New York University Tandon School of Engineering
Computer Science and Engineering

CS-GY 6033: Written Homework 2.
Due Thursday, Feb. 19, 2026, 11:59pm.

Collaboration is allowed on this problem set, but solutions must be written-up individually. If we suspect that you copied your solution directly from AI, we will set up a meeting with you where you will need to explain your solution. If you fail to do so, you will receive zero points for the homework.

Problem 1: Recurrence

Determine the worst case time complexity of each of the recursive algorithms below. In each case, state the recurrence relation describing the runtime. Solve the recurrence relation, either by unrolling it or showing that it is the same as a recurrence we have encountered in lecture.

1.
```python
import math
def find_max(numbers):
    """Given a list, returns the largest number in the list.

    Remember: slicing a list performs a copy, and so takes linear time.
    """
    n = len(numbers)
    if n == 0:
        return 0
    if n == 1:
        return numbers[0]
    mid_left = math.floor(n / 3)
    mid_right = math.floor(2n / 3)

    return max(
        find_max(numbers[:mid_left]),
        find_max(numbers[mid_left:mid_right])
        find_max(numbers[mid_right:])
    )
```

2. In this problem, remember that // performs *flooring division*, so the result is always an integer. For example, 1//2 is zero. *random.randint(a,b)* returns a random integer in $[a, b)$ in constant time. Note that you are asked to determine the **worst-case** time complexity of the following algorithm.

```python
import random
def foo(n):
    """This doesn't do anything meaningful."""
    if n == 0:
        return 1

    # generate n random integers in the range [0, n)
    numbers = []
    i = 0
    while i < n:
        i = i + 2
        number = random.randint(1, n)
        numbers.append(number)

    x = sum(numbers)
```

```
if x is even:
    return foo(n//2) / x**.5
else
    return foo(n//2) * x
```

3. Consider a recurrence of the form $T(n) = aT(n/b) + f(n)$, where $a > 0$, $b > 1$, and $f(n)$ is an asymptotically positive function. Determine non-trivial sufficient conditions on a, b, and $f(n)$ under which the solution satisfies $T(n) = \Theta(f(n))$. You may assume n is a power of b. Justify your answer.

Problem 2: Rotated Sorted Array

A *rotated sorted array* is an array that is the result of taking a sorted array and moving a contiguous section from the front of the array to the back of the array. For example, the array $[5, 6, 7, 1, 2, 3, 4]$ is a rotated sorted array: it is the result of taking the sorted array $[1, 2, 3, 4, 5, 6, 7]$ and moving the first 4 elements, $[1, 2, 3, 4]$, to the back of the array. Sorted arrays are also rotated sorted arrays, technically speaking, since you can think of a sorted array as the result of taking the sorted array and moving the first 0 elements to the back.

For example, all rotated version of $[1, 2, 3, 4, 5]$ is the following:

- $[1, 2, 3, 4, 5]$
- $[5, 1, 2, 3, 4]$
- $[4, 5, 1, 2, 3]$
- $[3, 4, 5, 1, 2]$
- $[2, 3, 4, 5, 1]$

Write an algorithm to find the value of the minimum element in a rotated sorted array. It is given the array *arr* and the indices *start* and *stop* which indicate the range of the array that should be searched. You may assume the numbers in *arr* are **unique**. Your algorithm should have time complexity $\Theta(\log n)$.

Problem 3: Removing Outliers

Suppose you are trying to remove outliers from a data set consisting of points in $\mathbb{R}^d$. One of the simplest approaches is to remove points that are in "sparse" regions – that is, points that don't have many other points close by. To do this, we might calculate the distance from a point to it's kth closest neighbor. If this distance is above some threshold, we consider the point an outlier.

More generally, the task of finding the distance from a query point to its kth closest "neighbor" is a common one in data science and machine learning. Here, we'll consider the 1-dimensional version of the problem of finding kth neighbor distance. In a file named `knn_distance.py`, write a function named `knn_distance(arr, q, k)` that returns a pair of two things:

- the distance between q and the kth closest point to q in *arr*;
- the kth closest point to q in *arr*

The query point q does not need to be in *arr*. For simplicity, *arr* will be a Python list of numbers, and q will be a number. k should start counting at one, so that *knn_distance(arr, q, 1)* returns the distance between q and the point in *arr* closest to q. Your approach should have an expected time of $\Theta(n)$, where n is the size of the input list. Your function may modify *arr*. In cases of a tie, the point you return is arbitrary (though the distance is not). Your code can assume that k will be $\leq len((arr)$.

Example:

```
>>> knn_distance([3, 10, 52, 15], 19, 1)
(4, 15)
>>> knn_distance([3, 10, 52, 15,], 19, 2)
(9, 10)
>>> knn_distance([3, 10, 52, 15], 19, 3)
(16, 3)
```

As this is a programming problem, submit your code to the Gradescope autograder.

Problem 4: BST with Floor and Ceiling

In a file named `bst_with_fcd.py`, create a class named `BinarySearchTree` which implements a binary search tree with the following methods and attributes:

- `.root`: the root node of the tree; a Node object. If the tree is empty, this should be `None`.
- `.insert(key)`: insert a new node with the given key. Should take $O(h)$ time. Should allow duplicate keys. This method should return a `Node` object representing the node.
- `.delete(node)`: remove the given `Node` from the BST. Should take $O(h)$ time.
- `.query(key)`: return the `Node` object with the given key, if it exists; otherwise raise `ValueError`. Should take $O(h)$ time.
- `.floor(key)`: return the `Node` with the largest key which is $\leq$ the given key. If there is no such key, raise `ValueError`. Should take $O(h)$ time.
- `.ceil(key)`: return the `Node` with the smallest key which is $\geq$ the given key. If there is no such key, raise `ValueError`. Should take $O(h)$ time.

Problem 5: Online Median

In a file named `online_median.py`, create a class named `OnlineMedian` which stores a collection of numbers and has which has two methods: `.insert(x)`, which inserts a number into the collection in $O(\log n)$ time, and `.median()` which computes the median of all numbers inserted so far in $\Theta(1)$ time.

If no numbers have been inserted so far, `.median()` should return `NaN`.

Problem 6: Quicksort Expected Time Complexity

The time complexity of both the `quickselect` and `quicksort` algorithms depends on how lucky we are in selecting a pivot. Using a similar argument to the one we saw in lecture for the time complexity of `quickselect`, show that the expected time complexity of the `quicksort` algorithm is $\Theta(n \log n)$.

```
def quicksort(arr, start, stop):
    """Sort arr[start:stop] in-place."""
    if stop - start > 1:
        pivot_ix = random.randrange(start, stop)
        pivot_ix = partition(arr, start, stop, pivot_ix)
        quicksort(arr, start, pivot_ix)
        quicksort(arr, pivot_ix+1, stop)
```